Chen *et al. Cybersecurity* (2026) 9:215
https://doi.org/10.1186/s42400-026-00653-9

Cybersecurity

RESEARCH Open Access

A deep learning based side-channel attack on BIKE

Geng Chen[1], Yanbin Li[1*], Shilin Sun[1], Zongyue Wang[2], Chunpeng Ge[1], Fusheng Wu[3] and Qiuliang Xu[1]

Abstract

Post-quantum cryptographic implementations are increasingly exposed to side-channel threats, making implementation-level security assessment an essential complement to algorithmic analysis. BIKE is a representative code-based key encapsulation mechanism, and its latest specification adopts the Black-Gray-Flip (BGF) decoder, which offers practical advantages in decoding performance and implementation regularity. However, the side-channel security of this decoding scheme has not been adequately studied in the latest BIKE reference implementation. In this work, we investigate this problem and develop a deep-learning-based side-channel attack for recovering a private-key-dependent sequence from long BIKE decoding traces. Firstly, we examine the BGF decoding procedure in the latest BIKE reference implementation and show that its repeated counter computations and threshold-based bit-flipping decisions give rise to exploitable side-channel leakage. Secondly, we transform BIKE BGF decoding leakage recovery into an alignment-free sequence-to-sequence task and instantiate it with a convolutional recurrent neural network (CRNN) trained using Connectionist Temporal Classification (CTC). This formulation enables direct recovery of the target sequence from complete traces without relying on manually defined temporal boundaries. Finally, we evaluate the proposed formulation through end-to-end recovery experiments, ablation analysis, and comparison with representative baselines. In the evaluated profiled, unprotected, same-platform ARM Cortex-M4 setting, experimental results show that the proposed method achieves 99.69% bit accuracy and 95.68% perfect sequence recovery on the independent test set. Our target is a 16-bit sequence of counter-comparison outcomes generated during BIKE decoding; this sequence is a private-key-dependent intermediate rather than the complete private key. Complete private-key recovery would require accumulating such observations over multiple executions and applying an additional post-processing stage. Overall, these results show that alignment-free sequence recovery is effective for BIKE side-channel analysis on long, temporally unstable traces in this evaluated setting.

Keywords Side-channel attack, Post-quantum cryptography, BIKE, Deep learning, Sequence translation, Connectionist temporal classification

Introduction

Background and motivation

As the standardization and deployment of post-quantum cryptography continue to advance, implementation security has become increasingly important alongside theoretical security. Even when a cryptographic construction is mathematically secure, its physical implementation may still leak secret-dependent information through power consumption, electromagnetic radiation, or timing behavior (Kocher et al. 1999; Delgado-Vargas et al. 2024). Such leakage creates opportunities for side-channel

*Correspondence:
Yanbin Li
yanbinli@sdu.edu.cn
[1] School of Software, Shandong University, 1500 Shunhua Road, High-tech Zone, Jinan 250101, China
[2] Open Security Research, Inc., Shenzhen, China
[3] Guizhou Key Laboratory of Economic System Simulation, Guizhou University of Finance and Economics, Guiyang 550025, China

© The Author(s) 2026. **Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

attacks, which have long been recognized as a major threat to practical cryptographic systems. Therefore, for post-quantum schemes intended for real-world deployment, evaluating resistance against physical leakage is now an essential part of security assessment (Delgado-Vargas et al. 2024).

BIKE is a code-based post-quantum key encapsulation mechanism built from Quasi-Cyclic Moderate-Density Parity-Check (QC-MDPC) codes and submitted to the National Institute of Standards and Technology (NIST) Post-Quantum Cryptography standardization process (Aragon et al. 2024). Prior studies have shown that both software and hardware implementations of BIKE are vulnerable to practical side-channel key-recovery attacks (Cheriere et al. 2023; Beckwith et al. 2024; Guo et al. 2022). BIKE is therefore a meaningful target for studying implementation security in post-quantum cryptography. In particular, the decapsulation and decoding stages of BIKE may show different execution patterns for different inputs, which can lead to distinguishable side-channel leakage in practical measurements.

In the latest BIKE specification, the recommended decoder is the BGF decoder (Aragon et al. 2024; Sendrier and Vasseur 2019). This decoder is attractive in practice because it combines a fixed-number-of-steps structure with favorable decoding-failure behavior, and is therefore well suited to secure and efficient decapsulation. However, from the perspective of side-channel analysis, the operational pattern of the decoder also introduces a distinctive challenge. During iterative decoding, the implementation repeatedly performs counter computation, threshold comparison, and conditional bit flipping (Richter-Brockmann et al. 2020). Because the counter-computation function is invoked repeatedly during one decoding execution, multiple private-key-dependent decision points can leave leakage in the same measured waveform. Consequently, the useful leakage is spread across the trace rather than confined to fixed short intervals, and the location and duration of each informative region may change with the evolving decoding state.

Under such conditions, recovering multiple target bits from a long trace is no longer simply a matter of applying local classification on pre-defined fixed windows. Even when the overall trace length is kept constant, the informative segments associated with different bits may shift, stretch, or overlap in time, making the correspondence between local waveform regions and target labels unstable (Cagli et al. 2017). This is precisely the setting in which rigid segmentation becomes inadequate. For this reason, we do not treat the studied BIKE decoding leakage-recovery task as a conventional fixed-window classification problem. Instead, the main methodological contribution is to formulate BIKE BGF decoding leakage recovery as an alignment-free sequence modeling task from a long physical waveform to a short target bit sequence, and to validate this formulation using a practical CRNN-CTC instantiation in the profiled, unprotected, same-platform setting evaluated in this work (Maghrebi et al. 2016; Lu et al. 2021).

Gap in existing studies

Existing studies have already shown that BIKE implementations are vulnerable to side-channel key-recovery attacks, but the attack settings considered so far remain relatively specific. Cheriere et al. (2023) studied a software implementation of BIKE and showed that secret information can be recovered by combining power-consumption analysis with information-set decoding. Beckwith et al. (2024) further demonstrated that BIKE is also vulnerable at the hardware level, and reported full key recovery from a public hardware implementation with only several dozen traces. Taken together, these studies establish the practical side-channel exposure of BIKE. Meanwhile, their attacks on BIKE were all targeted at specific implementation versions or versions specific to certain hardware conditions, and did not attack the latest reference implementation of BIKE. More specifically, there was no method to directly reconstruct the desired sequence from the acquired curve segments.

The main limitation of the existing literature lies elsewhere. Prior attacks are typically built on leakage features that appear at relatively stable temporal locations and can therefore be linked to specific target intermediates. In other words, they still rely, either explicitly or implicitly, on the attacker being able to identify informative segments or to construct an analysis pipeline around reasonably well-localized points of interest. Such assumptions are acceptable in controlled acquisition settings, but they become much harder to maintain when traces are collected in a fixed capture window and contain timing jitter, silent intervals, and data-dependent duration variation.

Under these conditions, the difficulty is not simply that local classification becomes less accurate. More fundamentally, the correspondence between waveform segments and target labels is itself unstable. Once this correspondence is no longer stable, the conventional pipeline based on segmentation, local classification, and sequence concatenation becomes unreliable, since the predefined boundaries no longer coincide with the actual information-bearing intervals.

Accordingly, the main limitation of the existing literature lies not merely in attack performance, but in the way the recovery task is formulated. In particular, insufficient attention has been paid to the case where BIKE key recovery must be performed from long traces without

reliable temporal boundaries. Under such conditions, it is more appropriate to transform the problem into an alignment-free mapping from a long physical trace to a short target bit sequence, where the correspondence between trace patterns and target labels is learned implicitly from data rather than imposed a priori through manual segmentation. This formulation underlies our method and differentiates it from prior BIKE attacks.

Research objective

This study focuses on end-to-end recovery of a private-key-dependent target sequence from long side-channel traces generated during BIKE decapsulation. In BGF decoding, the implementation repeatedly computes counters and compares them with decoding thresholds to determine whether candidate bits should be flipped. The target considered in this work is the 16-bit sequence formed by these counter-threshold comparisons. The goal is to determine whether a measured physical waveform can be mapped directly to this sequence when traces are long, contain silent intervals, and lack reliable segment boundaries. Accordingly, we formulate BIKE decoding leakage recovery as an alignment-free sequence modeling task, reducing the reliance on manual segmentation and point-of-interest (POI) selection.

On this basis, the study addresses two related questions. First, we examine whether deep temporal modeling can replace the conventional analysis pipeline based on trace alignment and fixed local partitioning when frame-level boundary information is unavailable, thereby enabling direct recovery of the target bit-flipping sequence from long traces. Second, within the evaluated profiled, unprotected, same-platform setting, we examine whether an end-to-end framework with implicit alignment capability can better tolerate boundary mismatch and improve recovery performance relative to methods based on rigid segmentation under non-uniform duration and silent-interval interference. To this end, we use the CRNN-CTC pipeline as a validation vehicle for the alignment-free sequence-recovery formulation, rather than presenting CNNs, BiLSTMs, or CTC as newly introduced neural-network modules.

Contributions

The main contributions of this paper are as follows:

- This work presents a side-channel attack against the BIKE reference implementation corresponding to the fourth-round standardization version. By focusing on the current decoding procedure adopted in the reference code, we show that the implementation remains vulnerable to leakage exploitation in the evaluated profiled, unprotected, same-platform long-trace acquisition setting.
- We transform BIKE BGF decoding leakage recovery from long traces into an alignment-free sequence-to-sequence problem. This formulation directly maps a measured waveform to the corresponding 16-bit `ctr` function sequence and avoids imposing manually selected fixed windows or frame-level boundaries. The recovered sequence is a private-key-dependent intermediate, not the complete BIKE private key.
- We validate this alignment-free recovery formulation using a CRNN-CTC instantiation composed of established CNN, BiLSTM, and CTC components. The experimental results show that the method achieves 99.69% bit accuracy and 95.68% perfect sequence recovery in the evaluated profiled, unprotected, same-platform setting. These results show that alignment-free sequence recovery is effective for the studied BIKE BGF decoding traces with long and temporally unstable leakage.

Paper organization

The remainder of this paper is organized as follows. Section "Preliminaries" introduces the preliminaries used throughout the paper, including the notations, the BIKE scheme, and the application of deep learning to side-channel analysis. Section "Threat model and problem formulation" presents the threat model and problem formulation, including the attack setting, the trace deformation caused by fixed-window acquisition, and the alignment-free sequence translation formulation. Section "CRNN-CTC attack" describes the proposed CRNN-CTC attack framework, including the overall pipeline, the deep bidirectional long short-term memory(BiLSTM) module, and the CTC-based alignment-free optimization and decoding strategy. Section "Experiments" reports the experimental setup and results, including the ablation study and the comparison with baselines based on rigid segmentation. Section "Discussion" discusses the cryptographic meaning of the recovered 16-bit sequence and its relationship to full BIKE private-key recovery. Finally, Sect. "Conclusion" concludes the paper.

Preliminaries

Notations

Let $\mathbb{F}_2$ denote the binary finite field and $\mathcal{R}$ denote the cyclic polynomial ring $\mathbb{F}_2[X]/(X^r - 1)$, where r is a prime number. The BIKE scheme is parameterized by a tuple (r, w, t), where r is the block length, w is the row weight of the parity-check matrix, and t is the weight of the error vector. For any binary polynomial $g \in \mathcal{R}$, $|g|$ denotes its Hamming weight.

The private key space $\mathcal{H}_w$ is defined as the set of pairs $(h_0, h_1) \in \mathcal{R}^2$ such that $|h_0| = |h_1| = w/2$. Similarly, the error space $\mathcal{E}_t$ is defined as the set of vectors $(e_0, e_1) \in \mathcal{R}^2$ where the total weight $|e_0| + |e_1| = t$.

BIKE scheme

BIKE is a Key Encapsulation Mechanism (KEM) based on Quasi-Cyclic Moderate-Density Parity-Check (QC-MDPC) codes and was submitted to the NIST Post-Quantum Cryptography standardization project. Its security is commonly related to the hardness of the quasi-cyclic variants of the Syndrome Decoding (SD) and Codeword Finding (CF) problems. In the specification, the scheme is built upon the random oracles H, K, and L, which are concretely instantiated using SHAKE256 and SHA384. At the algorithmic level, BIKE consists of three main procedures:

- *Key Generation* This procedure samples the private key (h_0, h_1) uniformly at random from $\mathcal{H}_w$. The public key is then computed as $h = h_1 h_0^{-1} \in \mathcal{R}$.
- *Encapsulation* Given the public key h, the sender samples a random message $m \in \mathcal{M}$ and derives a sparse error vector $(e_0, e_1) \leftarrow H(m, \pi_l(h))$. The ciphertext is then formed as $c = (c_0, c_1)$, where $c_0 = e_0 + e_1 h \in \mathcal{R}$ and $c_1 = m \oplus L(e_0, e_1)$. The shared secret is finally obtained as $K(m, c)$.
- *Decapsulation* Upon receiving the ciphertext $c = (c_0, c_1)$, the receiver first computes the syndrome-related quantity $c_0 h_0$ and then invokes the decoder to recover the error vector, namely $e' \leftarrow \mathit{decoder}(c_0 h_0, h_0, h_1)$. The candidate message is reconstructed as $m' = c_1 \oplus L(e')$, after which a consistency check $e' = H(m', \mu)$ is carried out before the shared secret is returned.

Among the three procedures, decapsulation is the most relevant stage for the present study, because its success depends directly on the recovery of the error vector from the syndrome. In the current BIKE specification, the recommended decoder is the **BIKE-Flip** decoder, which runs for a fixed number of iterations *NbIter* in order to reduce timing leakage at the coarse execution level. In each iteration, the decoder evaluates every bit position j by computing a counter that measures the number of unsatisfied parity-check equations associated with that position. In implementation, this operation is typically realized through a weight-counting routine, often denoted as `ctr`. A bit e_j is flipped whenever the corresponding counter is no smaller than a threshold T, where T is not constant but depends on the initial syndrome weight, the current syndrome weight, and the iteration index. As a result, although the decoder follows a fixed high-level iteration schedule, its internal operations remain strongly data dependent. This dependence is reflected in conditional updates, counting operations, and state transitions, which in turn make the decoding stage the principal source of side-channel leakage considered in this work.

Deep learning for side-channel analysis

Deep learning has become the dominant paradigm in profiled side-channel analysis over the past several years. Its central advantage is the ability to learn discriminative representations directly from raw or lightly processed traces, which substantially reduces the reliance on handcrafted points of interest and explicit statistical modelling. A comprehensive systematisation of the field, covering attack formulations, architectures, and open challenges, is provided by Picek et al. (2023).

The application of deep learning to profiled side-channel analysis was pioneered by Maghrebi et al. (2016), who evaluated convolutional, recurrent, and fully connected architectures against both unprotected and masked AES implementations and showed that all three consistently outperformed the classical template attack. To support reproducibility and cross-study comparison, Benadjila et al. (2020) introduced the ASCAD benchmark database, an open collection of electromagnetic traces from a masked AES implementation together with reference neural network architectures. ASCAD established a common evaluation basis and has since been widely adopted as the standard benchmark in the field.

CNNs have been the most widely adopted model class in this context, primarily because their local receptive fields and weight-sharing scheme make them sensitive to short-range leakage patterns while remaining relatively tolerant of clock jitter, small timing offsets, and other desynchronisation effects. Cagli et al. (2017) demonstrated that a CNN combined with data augmentation can mount a successful profiling attack on traces misaligned by both software and hardware countermeasures, without any prior trace realignment or manual point-of-interest selection. Kim et al. (2019) further showed that artificially adding Gaussian noise to the training input acts as a regulariser and consistently improves CNN attack performance across datasets with varying levels of noise and countermeasures. To move beyond empirical architecture selection, Zaid et al. (2020) proposed a theoretical methodology for designing efficient CNN architectures in the profiling context, using visualisation techniques to explain the role of each hyperparameter during the feature selection phase and providing guidelines for setting filter sizes with respect to the degree of desynchronisation present in the traces.

Recurrent architectures are better suited to scenarios in which informative leakage is spread over a longer time span. Maghrebi (2019) showed that LSTM networks outperform CNN and MLP in attacking secret-sharing implementations, where the relevant leakage comes from multiple shares residing at different positions in the trace—a setting that requires retaining context across many time steps rather than recognising a fixed local pattern. The ability to propagate information through gated hidden states, rather than convolving a fixed-length window over the input, gives LSTM an advantage whenever the temporal distance between related leakage events is large or variable. Beyond the profiled setting, Timon (2019) introduced a non-profiled deep learning attack, demonstrating that the translation-invariance property of CNNs can be exploited against desynchronised traces even without access to a labelled profiling set, by combining key guesses with deep learning training metrics as a distinguisher.

In practice, the two architectures are complementary. CNNs extract stable local features and compress the temporal dimension of the trace; stacking a recurrent module on top allows the model to reason over the resulting feature sequence at a coarser granularity. This combination forms the architectural basis of the framework proposed in this work, and its suitability for the BIKE side-channel setting—where data-dependent execution paths produce leakage at irregular and non-uniform positions—is discussed in detail in Sect. "CRNN-CTC attack".

Threat model and problem formulation

Attack setting and adversarial capability

We adopt a standard profiling-attack setting, in which the attack pipeline consists of an offline modeling stage and an online recovery stage on the same ARM Cortex-M4 platform and unprotected BIKE implementation. In the offline stage, the adversary collects power traces from a controlled device and obtains the corresponding labels to train an end-to-end sequence-recovery model. In the online stage, the adversary observes traces from the target device and infers the target sequence from those traces only. The reported results should therefore be interpreted as same-platform profiled-attack results, rather than evidence of robustness across devices, acquisition sessions, implementations, keys, or countermeasures.

Regarding adversarial capability, we assume that the attacker can record raw power waveforms during BIKE-related key-processing operations and has knowledge of the algorithmic implementation and execution flow at a functional level. However, the attacker does not rely on manually annotated points of interest (POIs) or precise trigger-based alignment. In other words, the attacker can acquire long traces within a fixed capture window, but cannot obtain reliable frame-level boundaries. This constraint specifies the acquisition setting considered in this work; robustness across devices, sessions, implementations, keys, or protected executions lies beyond the present scope.

The attack objective is defined as follows: given an input trace x, recover the corresponding 16-bit target sequence $l = (b_0, \ldots, b_{15})$. This target sequence is not an arbitrary label and is not the error vector itself. It records the first 16 binary values used by the counter computation in the BIKE decoding implementation. More precisely, with the zero-based loop index $i = 0, \ldots, 15$, the i-th target bit is

$$b_i = s\left[(\texttt{h_compact_col}[i] + \texttt{position}) \bmod r\right], \quad (1)$$

and the supervised sequence is $l = (b_0, \ldots, b_{15})$. Here, s is the syndrome-related vector, `h_compact_col` stores the support indices associated with the private-key polynomial h_0, and `position` is the currently evaluated decoder position. Therefore, the recovered 16 bits are secret-indexed samples of s used by the private-key-dependent counter computation. In evaluation, we report both bit-level and sequence-level metrics. The former measures local discriminative performance for individual bits, while the latter measures one-shot recovery of the full target sequence. Compared with local classification accuracy alone, sequence-level recovery is a more direct indicator of compromise within the evaluated attack setting.

Cryptographically, these recovered bits are relevant because they sample the syndrome at positions determined by the secret support. A single recovered 16-bit sequence does not by itself reveal the full BIKE private key. Its concrete attack advantage is that it acts as a side-channel leakage oracle for secret-indexed syndrome samples. Under a chosen-ciphertext setting, an adversary can choose $c_0 = X^k$, so the syndrome-related vector becomes $s = c_0 h_0 = X^k h_0$, i.e., a cyclic rotation of the private-key polynomial. Repeating target-sequence recovery over multiple choices of k and decoder positions accumulates combinatorial constraints on the unknown support encoded by `h_compact_col`. Such accumulated partial observations can then be combined with information-set decoding or combinatorial post-processing, as in prior BIKE attacks (Cheriere et al. 2023; Guo et al. 2022), to move toward full private-key recovery. In this paper, we evaluate the core side-channel sequence-recovery step; we do not claim that the neural network alone outputs the complete BIKE private key, and full private-key reconstruction from accumulated observations is left as future work.

Overall, this work targets end-to-end recovery of a private-key-dependent counter-derived bit sequence under a profiled, unprotected, same-platform attack condition

where long traces are observable but explicit alignment information is unavailable. This setting provides the basis for the deformation analysis in Sect. "Duration gap" and the formal problem definition in Sect. "Sequence translation formulation".

Duration gap

Under a fixed hardware capture window, equal trace length does not imply semantic alignment. In BIKE implementations, different data paths often incur different execution durations, so the location and concentration of informative leakage are not stable within the same time window. Our measurements during BIKE decoding show that, in the `ctr` function, waveform segments associated with target bits "0" and "1" are typically unequal in duration. This behavior matches the semantics of the recovered branch outcomes: when $b_i = 1$, the corresponding condition is satisfied and the conditional branch is taken, so the associated counter is incremented; when $b_i = 0$, the condition fails and this update is skipped. In our traces, the branch-taken case occupies about 670 sample points, whereas the skipped case occupies about 200 sample points. Because s behaves close to a random binary vector in the profiled traces, the target bits are close to balanced and the Hamming-weight distribution follows the behavior expected from random sampling. As a result, traces with the same number of sampling points may still carry discriminative information over different temporal intervals.

This observation has direct consequences for rigid segmentation. The rigid strategy assumes that uniformly partitioning a long trace into fixed windows yields semantically consistent units. That assumption is difficult to maintain when 0-related and 1-related segments are intrinsically unequal. Informative transitions can be truncated or assigned to adjacent windows, which introduces structural label mismatch. The mismatch is not random measurement noise but a systematic effect induced by duration non-uniformity, and the resulting error accumulates through both classification and decoding stages.

The same reasoning also explains why simply scaling up local feature extractors is insufficient. When upstream window boundaries do not match the true information-bearing intervals, even a stronger local classifier is trained on misaligned inputs and cannot establish a stable global mapping. The central issue in this section is therefore not the absence of learnable local patterns but the lack of reliable temporal correspondence between features and target labels.

Accordingly, we characterize the difficulty as a lack of implicit alignment. The adversary can observe full long traces, yet cannot reliably construct a frame-to-bit correspondence. This motivates the formal treatment in the next subsection. Instead of a boundary-dependent window-wise classification paradigm, we formulate key recovery as a long-waveform to short-symbol sequence-to-sequence mapping task without explicit alignment constraints.

Sequence translation formulation

Under the attack setting defined above, the input and output lengths are inherently mismatched. For each sample, the attacker observes a long power trace $x = (x_1, \ldots, x_T)$, while the target is a short bit sequence $l = (l_1, \ldots, l_U)$ with $U = 16$ in our case. Because 0-related and 1-related waveform segments are not equal in duration, local time indices in the trace cannot be mapped to target bits with a fixed boundary rule. For this reason, the recovery task is better posed as conditional sequence modeling rather than window-wise classification.

We therefore define the abstract recovery objective as learning the conditional probability $p(l \mid x)$ and perform inference by

$$l^* = \arg\max_{l} p(l \mid x) \tag{2}$$

This definition supervises the whole sequence and does not require frame-level labels or manually annotated points of interest. In practical terms, the model must infer an implicit correspondence between a long physical waveform and a short string of `ctr` branch outcomes during training.

This abstract formulation specifies what the model must learn—a direct mapping from a long waveform to a short symbol sequence without committing to how the alignment is realized. The concrete CTC instantiation, including the extended alphabet, blank symbol, path marginalization, and decoding rule, is deferred to Sect. "CTC-based alignment-free sequence optimization and decoding", where it is introduced together with the network architecture.

CRNN-CTC attack

Overview of the CRNN-CTC

The CRNN-CTC pipeline is used here as an implementation of the alignment-free recovery formulation, not as a claim that CNNs, BiLSTMs, or CTC are new neural components. Its role is to provide a practical way to test whether sequence recovery without explicit boundaries can work for BIKE BGF decoding traces with long and unstable temporal structure. We begin with lightweight preprocessing of the aligned BIKE power traces, applying band-pass filtering, Z-score normalization and downsampling to reduce spatiotemporal complexity. Subsequently, a multi-layer 1D-CNN is employed as the front-end feature extractor. Through successive convolution and

pooling operations, it distills local power-consumption signatures into high-level representations while simultaneously compressing the trace along the temporal dimension. The resulting high-dimensional feature sequence is passed to a BiLSTM network. Although the traces are globally aligned, informative leakage components are often dispersed across long temporal spans. The BiLSTM captures these long-range dependencies, suppresses key-independent activity, and integrates distributed local leakage cues. Finally, we use CTC to bridge the substantial length mismatch between the input traces and the target 16-bit sequence while accommodating the absence of frame-level labels. During training, the CTC objective marginalizes over all valid alignment paths, enabling end-to-end optimization without manual POI annotation. During inference, greedy decoding merges repeated states and removes blank tokens, producing the target 16-bit sequence directly from long waveform traces.

Dependency modeling via deep BiLSTM

After local waveform features are extracted by the front-end 1D-CNN and the temporal resolution is reduced through downsampling, the original one-dimensional signal is converted into a multichannel feature sequence. Nevertheless, due to the locality of convolutional receptive fields, CNNs alone are insufficient for capturing long-range dependencies across temporally distant waveform segments. Therefore, a deep bidirectional recurrent network is introduced on top of the convolutional encoder to perform global sequence modeling.

The recurrent module estimates a local label distribution at each time step x_t for the input feature sequence $x = (x_1, \ldots, x_T)$. We use recurrent neural networks (RNNs) because they model long-range temporal dependencies more effectively than point-wise classification, which is important for noisy side-channel waveforms. They also support backpropagation through time to the front-end convolutional layers so that feature extraction and temporal modeling can be trained jointly in an end-to-end manner. In addition, RNNs naturally handle variable-length waveform sequences and therefore avoid the fixed-length input constraint.

A standard RNN maintains a hidden state h_t that is updated at each time step by combining the current input x_t with the previous state h_{t-1} through a learned nonlinear transition. Although this recurrence gives the network access to historical context, gradient signals attenuate exponentially as they propagate backward through many steps, so the effective memory of a vanilla RNN is in practice far shorter than the sequence itself.

LSTM units address this directly by introducing a dedicated memory cell C_t alongside three multiplicative gates. The forget gate decides what fraction of the previous cell state to discard; the input gate determines how strongly the current input is written into C_t; and the output gate projects the updated cell state into the hidden representation h_t passed to the next layer. Because the cell state is updated additively rather than through repeated nonlinear compression, gradients can flow over hundreds of time steps without vanishing, which is essential when informative leakage is separated from other parts of the trace by long silent stretches or unrelated instruction activity.

A remaining limitation is that a unidirectional LSTM at time step t can only draw on observations up to x_t; it has no access to what follows. In power side-channel signals this is consequential: the magnitude of the consumption at a given clock cycle reflects not only the current instruction state but also pipeline effects from operations that have not yet been observed. A fragment occurring during a silent interval, for instance, may be uninterpretable in isolation but becomes identifiable once the transition that follows it is seen. We therefore replace the single-direction LSTM with a bidirectional variant (BiLSTM), which processes the sequence in both the forward and backward directions and concatenates the two hidden states at each step:

$$h_t^{(\ell)} = \left[\overrightarrow{h}_t^{(\ell)} ; \overleftarrow{h}_t^{(\ell)} \right] \tag{3}$$

In our implementation we stack two BiLSTM layers. The first layer operates directly on the compressed CNN features and extracts local transition patterns; its output sequence is passed wholesale to the second layer, which can then attend to longer-range structure built from those local features. The two-layer depth is a practical compromise: a single layer leaves long-range dependencies undermodelled on traces of length $T' \approx 2083$, while three or more layers add training cost without consistent gain in our preliminary experiments. The stacked design is particularly suited to BIKE traces in which cryptographic leakage is sparse and intermittent—the lower layer identifies candidate leakage segments, and the upper layer resolves their identities in the light of the surrounding sequence.

Formally, let the CNN output sequence be $H^{(0)} = (h_1^{(0)}, \ldots, h_{T'}^{(0)})$, where T' is the compressed temporal length after convolution and pooling. For the ℓ-th BiLSTM layer ($\ell \in \{1, 2\}$), the forward and backward hidden states are

$$\begin{aligned} \overrightarrow{h}_t^{(\ell)} &= \mathrm{LSTM}_{\rightarrow}^{(\ell)}\left(\mathrm{h}_{\mathrm{t}}^{(\ell-1)}, \overrightarrow{h}_{\mathrm{t-1}}^{(\ell)}\right) \\ \overleftarrow{h}_t^{(\ell)} &= \mathrm{LSTM}_{\leftarrow}^{(\ell)}\left(\mathrm{h}_{\mathrm{t}}^{(\ell-1)}, \overleftarrow{h}_{\mathrm{t+1}}^{(\ell)}\right) \end{aligned} \tag{4}$$

and the layer output is obtained by concatenation:

$$h_t^{(\ell)} = \left[\overrightarrow{h}_t^{(\ell)}; \overleftarrow{h}_t^{(\ell)}\right] \tag{5}$$

The stacked connection used in our implementation is therefore

$$\begin{aligned} H^{(1)} &= \mathrm{BiLSTM}^{(1)}\left(\mathrm{H}^{(0)}\right) \\ H^{(2)} &= \mathrm{BiLSTM}^{(2)}\left(\mathrm{H}^{(1)}\right) \end{aligned} \tag{6}$$

After the top layer, a linear projection and log-softmax are applied at each time step over the CTC output alphabet $\mathcal{V}'$, which is specified explicitly in Sect. "CTC-based alignment-free sequence optimization and decoding"

$$\begin{aligned} z_t &= Wh_t^{(2)} + b \\ y_k^t &= \log \frac{\exp(z_{t,k})}{\sum_{k' \in \mathcal{V}'} \exp(z_{t,k'})} \quad k \in \mathcal{V}' \end{aligned} \tag{7}$$

Here, y_k^t is the frame-level log-posterior consumed by the CTC objective in Sect. "CTC-based alignment-free sequence optimization and decoding". This explicit mapping from $H^{(0)} \to H^{(1)} \to H^{(2)} \to y^t$ clarifies why the Deep BiLSTM output can be directly used for alignment-free sequence optimization.

During training, error gradients are propagated along both the temporal axis and network depth using Back-Propagation Through Time (BPTT), enabling stable parameter updates and end-to-end optimization. With this deep BiLSTM encoder, the model can traverse redundant silent intervals and unrelated instruction activity in long traces while preserving temporal continuity of key leakage evidence, and it can project physically separated cryptographic features into a unified semantic space. Consequently, the network produces frame-level posterior distributions with stronger global consistency, providing reliable inputs for the CTC alignment and decoding stage described next and establishing the basis for sequence recovery without explicit boundary annotations.

CTC-based alignment-free sequence optimization and decoding

Following the abstract sequence-translation formulation in Sect. "Sequence translation formulation", this subsection gives the implementation-specific CTC realization used in our CRNN-CTC attack. As indicated by Eq. (7), the Deep BiLSTM outputs frame-level log-posteriors y_k^t at each compressed time step. These outputs are converted into a trainable alignment-free decoder by defining the extended label set $\mathcal{V}' = \mathcal{V} \cup \{\epsilon\}$, where ϵ is the blank symbol used to absorb redundant frames, silent intervals, and transitional states without key-related semantics.

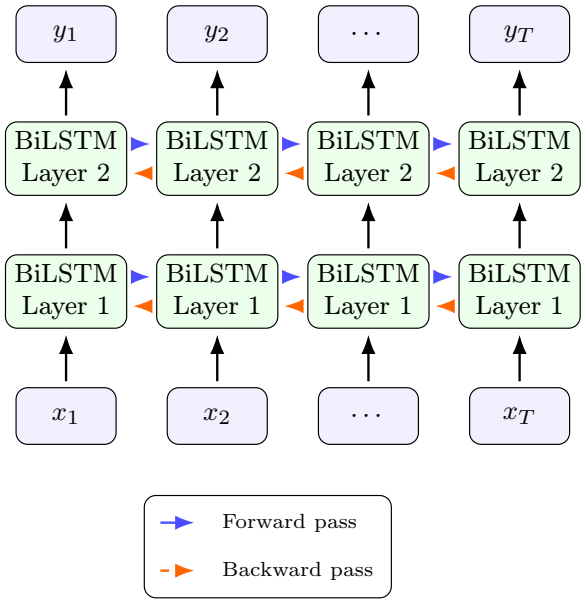


Fig. 1 Stacked Deep BiLSTM encoder. Solid blue arrows denote the forward pass; dashed orange arrows denote the backward pass. Vertical arrows indicate the feed-forward connections from inputs to Layer 1, from Layer 1 to Layer 2, and from Layer 2 to the output sequence

In our laboratory acquisition setting, we collected 21,200 oscilloscope traces from an unprotected ARM Cortex-M4 implementation in a profiled same-platform experiment; each raw trace has a fixed acquisition length of $T = 8{,}333$ sample points and corresponds to a 16-bit target sequence ($U = 16$) consisting of the first 16 `ctr` branch outcomes $s[(\texttt{h_compact_col}[i] + \texttt{position}) \bmod r]$. Therefore, the practical task in this evaluated setting is to recover 16 secret-indexed syndrome samples from 8333 points without frame-level boundaries. Empirical profiling shows that one bit "1" occupies about 670 points, while one bit "0" occupies about 200 points. Because these durations are highly unequal, leakage evidence is not uniformly distributed over time. For patterns such as "00001111", the early zeros may largely appear as silent/low-information regions, and the waveform may only show clear transitions around the later zero and subsequent ones. As a result, rigid fixed-window segmentation with frame-wise supervision is unreliable, whereas CTC can treat these non-informative or variable-duration regions as part of the latent alignment (Figs. 1, 2).

After introducing the extended alphabet $\mathcal{V}'$, CTC no longer relies on supervision from a single "ground-truth" segmentation path; instead, it marginalizes over all admissible alignments in probability space. Let the frame-level output sequence of the Deep BiLSTM have length T'. An alignment path is then defined as

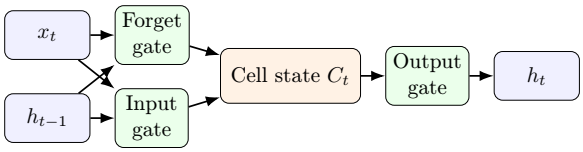


Fig. 2 Simplified structure of an LSTM unit used for temporal dependency modeling

$\pi = (\pi_1, \ldots, \pi_{T'}) \in \mathcal{V}'^{T'}$, i.e., a sequence with the same temporal length as the network output. Under the standard conditional-independence assumption across time steps, the probability of a specific path π conditioned on input feature sequence x is

$$P(\pi \mid x) = \prod_{t=1}^{T'} y_{\pi_t}^t \tag{8}$$

To map a relatively long alignment path π to the true 16-bit target sequence l, CTC defines a many-to-one mapping function $\mathcal{B}$. This function first merges consecutive repeated labels and then removes all blank symbols ϵ, producing a compressed output sequence. For example, $\mathcal{B}(\epsilon 11\epsilon 000\epsilon)$ collapses to the target fragment 10. This collapsing behaviour directly accommodates the two structural difficulties noted earlier. Since bit "1" occupies roughly 670 sample points and bit "0" only about 200, a rigid frame boundary cannot serve both faithfully; the blank symbol absorbs whatever surplus frames arise from the longer duration, while repeated labels cover the shorter ones, so both bit types are recovered through the same decoding rule without any duration-specific tuning. The conditional probability of the target sequence l is therefore obtained by summing over the entire preimage set:

$$p(l \mid x) = \sum_{\pi \in \mathcal{B}^{-1}(l)} p(\pi \mid x) \tag{9}$$

where $\mathcal{B}^{-1}(l)$ collects all paths that collapse to l under $\mathcal{B}$, not the inverse of any bijection.

Direct evaluation of this sum is intractable, as the number of valid paths grows exponentially with sequence length. CTC resolves this with a forward–backward recursion that factors the global sum into a product of local terms, bringing the computational cost down to $O(T'|\mathcal{V}'|)$ per training step. Minimising the negative log-likelihood $\mathcal{L}_{\text{CTC}} = -\ln p(l \mid x)$ then yields gradients for every output node y_k^t through standard BPTT, so the 1D-CNN front-end and the Deep BiLSTM are updated jointly without any manually annotated frame boundaries.

At inference time the forward–backward pass is replaced by greedy decoding: for each compressed time step t the label

$$\pi_t^* = \arg\max_{k \in \mathcal{V}'} y_k^t \tag{10}$$

is selected, and $\mathcal{B}$ is applied to the resulting path π^* to merge consecutive repeated labels and strip blank tokens, giving the final 16-bit target-sequence estimate $\hat{l} = \mathcal{B}(\pi^*)$. Greedy decoding scales linearly with T', which keeps batch evaluation tractable on long traces. More importantly, because the blank symbol absorbs timing fluctuations and silent execution intervals within the path itself, no explicit re-alignment step is needed between the oscilloscope capture and the sequence output—the decoder handles desynchronisation implicitly, consistent with the CTC objective used during training.

Algorithm 1 summarizes the complete procedure in our implementation, including forward–backward-based CTC training and explicit best-path (greedy) decoding during inference.

Algorithm 1 CTC training and best-path decoding for BIKE target-sequence recovery

Require: Trace batch X, sequence labels L, model f_θ, blank symbol ϵ
Ensure: Predicted target sequences $\hat{l}$ for each attack trace
$Z \leftarrow f_\theta(X)$ ▷ frame logits from 1D-CNN + Deep BiLSTM
$Y \leftarrow \log \mathrm{Softmax}(Z)$
Compute CTC loss $\mathcal{L}_{\mathrm{CTC}} = -\log P(L \mid X)$ via forward–backward recursion
Backpropagate through time and depth; update θ with Adam
for each attack trace x **do**
  $Y \leftarrow \log \mathrm{Softmax}(f_\theta(x))$
  Greedy decoding: $\pi_t^* \leftarrow \arg\max_{k \in \mathcal{V}'} Y_t(k)$ for $t = 1, \ldots, T'$
  Merge consecutive repeated labels in π^*; remove all blank symbols ϵ
  return predicted target sequence $\hat{l} = \mathcal{B}(\pi^*)$
end for

Experiments

Experimental setup and metrics

We collect traces on an ARM Cortex-M4 development board running an unprotected reference implementation of BIKE post-quantum cryptography. The experiments are conducted in a profiled same-platform setting: the training, validation, and test traces are acquired from the same implementation/platform class under the same laboratory acquisition setup. Leakage waveforms are captured in real time with a Keysight Technologies DSO-X 3024 T oscilloscope at a fixed sampling rate. After multiple rounds of automated acquisition, we obtain a side-channel dataset of 21,200 traces. The dataset is split into 19,000 training traces, 1000 validation traces (traces 19,001–20,000), and 1200 independent test traces (traces 20,001–21,200). The validation set is used for training monitoring and hybrid-template construction, while the independent test set is used only for final attack assessment. Each trace corresponds to one complete key-processing execution, has a fixed length of 8333 sample points, and is paired with a 16-bit ground-truth target sequence formed by the first 16 `ctr` branch-condition outcomes. The reported recovery rates are therefore computed on the 1200 independent test traces and characterize the profiled same-platform setting described above; robustness across devices, acquisition sessions, implementations, keys, and countermeasures remains outside the scope of the present evaluation.

For model training, we implement the proposed CRNN-CTC architecture in PyTorch and accelerate computation with an NVIDIA RTX 4090 GPU. We use Adam as the optimizer, with an initial learning rate of 0.001, batch size of 128, and 30 training epochs. The loss function is CTC loss (CTCLoss) with blank symbol id set to 2. In addition, because the input waveform sequences are long, we apply gradient clipping to mitigate gradient explosion during temporal backpropagation in the deep BiLSTM, with the gradient-norm threshold set to 5.0.

To quantify target-sequence recovery performance, we define two primary metrics: Bit Accuracy and Perfect Sequence Recovery Rate. Let A_{bit} denote Bit Accuracy. This metric measures the average correctness of individual branch-outcome bits over the test set and reflects the model's ability to decode local leakage features. It is computed as

$$A_{\mathrm{bit}} = \frac{\sum_{i=1}^{N} \sum_{j=1}^{L} \mathbb{I}(y_{i,j} = \hat{y}_{i,j})}{N \times L} \times 100\% \tag{11}$$

where N is the total number of test samples, $L = 16$ is the target-sequence length, and $\mathbb{I}(\cdot)$ is the indicator function that equals 1 when $\hat{y}_{i,j} = y_{i,j}$ and 0 otherwise. In contrast, Perfect Sequence Recovery Rate is a stricter sequence-level metric: a recovered sequence is counted as correct only when all 16 bits are predicted correctly; any single-bit error is treated as a failed recovery. It is defined as

$$A_{\mathrm{seq}} = \frac{1}{N} \sum_{i=1}^{N} \mathbb{I}\big(\forall j \in [1, L],\ y_{i,j} = \hat{y}_{i,j}\big) \times 100\% \tag{12}$$

In practical attack scenarios, a high Perfect Sequence Recovery Rate indicates that the side-channel sequence-extraction stage succeeds reliably; it should not be interpreted as a complete private-key recovery rate.

Ablation study on CRNN-CTC architecture

To examine the contribution of each key component in the end-to-end CRNN-CTC framework, we conduct a controlled ablation study with one factor changed at a time. The ablations cover data preprocessing with band-pass filtering and downsampling, the front-end local feature extractor based on 1D-CNN, and the global

Table 1 Ablation results of core modules in the CRNN-CTC framework

Exp	Configuration	Training time (s)	Bit accuracy (%)	Perfect sequence recovery (%)
Exp. 1	Proposed (full model)	83.3	99.69	95.68
Exp. 2	No band-pass filter	83.5	98.99	94.48
Exp. 3	No downsampling ($D=1$)	302.5	98.95	94.48
Exp. 4	No 1D-CNN module	261.6	93.86	84.49
Exp. 5	No Deep BiLSTM module	36.2	44.42	0.00

Proposed denotes the full model; each subsequent row removes one component while keeping all others fixed

temporal modeling module based on Deep BiLSTM. All settings are trained under the same hardware and software environment with identical hyperparameters, and all results are reported on the same independent test set.

Table 1 supports three main observations. First, preprocessing and temporal compression improve both effectiveness and efficiency. Removing the band-pass filter reduces Perfect Recovery from 95.68 to 94.48%. Removing downsampling increases training time from 83.3 to 302.5 s and also leads to a slight drop in recovery performance. This suggests that downsampling is useful not only for reducing computation, but also for keeping CTC training stable on long sequences. Second, the 1D-CNN module plays an important role in local feature extraction and temporal compression. Without it, Perfect Recovery drops to 84.49% and training time increases markedly, indicating that passing raw long sequences directly to the recurrent layers is both less efficient and less effective. Third, the Deep BiLSTM module is critical for temporal modeling. When it is removed, Bit Accuracy falls to 44.42% and Perfect Recovery drops to 0. These results indicate that convolutional features alone are not sufficient to capture the long-range temporal dependence present in this task. The specific contribution of the CTC alignment mechanism relative to the encoder is further isolated in Sect. "Comparison with Baselines" through an additional No-CTC ablation (Exp. 8).

Comparison with Baselines

To assess the practical benefit of alignment-free sequence recovery for long BIKE traces, we compare the proposed CRNN-CTC approach with two rigid-segmentation baselines and one additional No-CTC ablation designed to isolate the specific contribution of the CTC mechanism from the contribution of the encoder.

Rigid-segmentation baselines (Exp. 6 and Exp. 7). In this setting, a long waveform containing U key bits is uniformly divided into U fixed windows, and each window is treated as the observation corresponding to one target bit. Although this strategy is straightforward, it relies on the assumption that each bit occupies an approximately fixed temporal span, an assumption that is often violated in practical BIKE executions.

In Exp. 6, the long trace is first partitioned into 16 fixed segments, and each segment is then processed by a CNN-based local classifier to obtain bit-wise scores. In parallel, Pearson-correlation template matching is applied to the same segments, and the final decision is obtained by fusing the CNN scores with the template-matching scores. This baseline represents a conventional pipeline that combines rigid segmentation with local feature extraction and statistical matching. In Exp. 7, the same 16 fixed segments are used as supervision units, but the classifier is replaced with a CRNN backbone. More specifically, the segmented sequence is fed into a CRNN model, and frame-level cross-entropy loss is used to train the network under the predefined segment-to-bit correspondence. This baseline is intended to show whether a stronger sequential backbone alone can compensate for the error introduced by fixed segmentation.

No-CTC ablation (Exp. 8). To further isolate the contribution of the CTC alignment mechanism from the contribution of the CNN-BiLSTM encoder, we introduce Exp. 8, which keeps the complete encoder structure (1D-CNN front-end and two-layer Deep BiLSTM) identical to Exp. 1 but replaces the CTC output head with a frame-wise cross-entropy classifier. Specifically, the compressed feature sequence of length T' is reduced to exactly 16 output frames via adaptive average pooling, and each frame is then independently classified as bit 0 or bit 1 using a fully connected layer trained with cross-entropy loss. This construction eliminates the blank token, path marginalisation, and implicit alignment learning of CTC while preserving the same encoder capacity, thereby making the comparison between Exp. 1 and Exp. 8 a direct measure of what CTC contributes beyond a strong temporal encoder. For completeness, we also report the full CRNN-CTC model as Exp. 1.

Table 2 Performance comparison between the proposed CRNN-CTC, two rigid-segmentation baselines (Exp. 6 and Exp. 7), and a No-CTC ablation (Exp. 8)

Exp	Decoding mechanism	Bit accuracy (%)	Perfect sequence recovery (%)
Exp. 6	Rigid segmentation + CNN/template fusion	65.80	0.32
Exp. 7	Rigid segmentation + frame-wise CRNN classification	77.47	5.60
Exp. 8	CRNN encoder (same as Exp. 1) + No-CTC (frame-CE)	87.94	15.27
Exp. 1	CRNN-CTC (ours)	99.69	95.68

Table 3 Comparison between the present work and the BIKE side-channel attack by Cheriere et al. (2023)

Work	Platform	Attack target	Recovery	Latest BIKE
Cheriere et al. (2023)	Cortex-M4	$c_0 h_0$ rotation leakage	Full private key	No
This work	Cortex-M4	BGF decoding (long-trace)	16-bit sequence	Yes

Table 2 supports the following observations. Rigid segmentation reaches a clear performance limit. The main reason is the non-uniform timing structure of practical traces, where informative transitions are interleaved with long silent intervals. Under fixed boundaries, these transitions are often split across neighboring windows or assigned to mismatched segments, and the resulting alignment error propagates throughout decoding. Consistent with this observation, Exp. 6 attains only 0.32% perfect recovery, suggesting that template-score fusion is insufficient to offset boundary mismatch. Although Exp. 7 adopts a stronger recurrent backbone, its perfect recovery remains 5.60% when constrained by the same rigid partitioning.

CTC provides substantial alignment benefit beyond the encoder. Exp. 8 uses exactly the same CNN-BiLSTM encoder as Exp. 1 but replaces CTC with a fixed uniform mapping (adaptive average pooling to 16 frames followed by frame-wise cross-entropy). Under this setting, bit accuracy drops from 99.69 to 87.94% and perfect sequence recovery falls sharply from 95.68 to 15.27%. This gap directly quantifies the contribution of the CTC alignment mechanism itself: when the encoder is held fixed, removing the blank token and path marginalisation causes an 80.41 percentage-point drop in perfect recovery. The result confirms that the performance advantage of the proposed method is not attributable solely to encoder capacity, but depends critically on the implicit temporal alignment that CTC learns from the data.

By contrast, CRNN-CTC avoids manually defined segment boundaries. Through the blank symbol and path marginalisation in CTC, the model learns input–label correspondence directly from data during training, achieving 95.68% perfect recovery. Overall, these results indicate that modeling side-channel sequence recovery as an alignment-free Seq2Seq problem is better suited to long traces with irregular temporal leakage, and that CTC is the component responsible for handling the temporal non-uniformity that the encoder alone cannot resolve.

Comparison with prior BIKE attack

To further clarify the position of our work, we compare it with the BIKE side-channel study of Cheriere et al. (2023), which also targets a Cortex-M4 implementation. Despite sharing the same hardware platform, the two works differ substantially in their attack target and scope of applicability.

As shown in Table 3, Cheriere et al. exploit rotation leakage arising from the private-key-dependent $c_0 h_0$ polynomial multiplication, combining subtrace clustering with combinatorial post-processing to achieve full private-key recovery. However, this leakage source is no longer present in the latest BIKE reference implementation, where the corresponding operation has been replaced by a constant-time routine. Our work targets the BGF decoding procedure of the current reference code and formulates recovery of the private-key-dependent counter-derived sequence as a direct sequence-to-sequence problem over long traces. Unlike the prior work, however, the demonstrated output of our neural pipeline is this 16-bit intermediate sequence rather than a complete private key.

Discussion

From 16-bit sequence to private-key recovery

This paper demonstrates the recovery of a 16-bit sequence from long BIKE BGF decoding traces. To make its significance precise, this section clarifies three points in turn: what the recovered sequence represents, why recovering it provides a concrete attack advantage, and what the scope of the present evaluation is.

We first describe what the sequence represents. The target of this work is not an arbitrary label, nor is it

the BIKE private key or the error vector directly. It is a sequence of binary counter-comparison outcomes produced during the BGF decoding loop. Concretely, in each iteration, the decoder evaluates a bit position j by computing a counter that measures the number of unsatisfied parity-check equations involving j; it then compares this counter against a data-dependent threshold T. The 16-bit sequence recovered in our experiments records the first 16 such branch outcomes. More precisely, with zero-based loop index $i = 0, \ldots, 15$, the i-th target bit is

$$b_i = s[(\texttt{h_compact_col}[i] + \texttt{position}) \bmod r], \tag{13}$$

where s is the syndrome-related binary vector, `h_compact_col` stores the support indices of the private-key polynomial h_0, and `position` is the currently evaluated decoder position. Thus, each recovered bit b_i is a *secret-indexed syndrome sample*: the index into s is determined by the private key support, so observing b_i yields one bit of s at a position that is secret.

We next explain why recovering this sequence gives a concrete attack advantage. Although a single 16-bit observation does not reveal h_0 or (e_0, e_1) directly, it provides a side-channel leakage oracle for secret-indexed positions of the syndrome. Under a chosen-ciphertext setting, an adversary can send ciphertexts of the form $c_0 = X^k$ for varying rotation offset k, making the syndrome-related vector equal to $s = c_0 h_0 = X^k h_0$, a cyclic rotation of the private-key polynomial. Each call to the decoding oracle then reveals samples of this rotated syndrome at positions selected by `h_compact_col`. Repeating the target-sequence recovery over multiple choices of k and decoder positions accumulates combinatorial constraints on the unknown private-key support. These accumulated partial observations can subsequently be combined with information-set decoding or combinatorial post-processing—as employed in prior BIKE side-channel attacks Cheriere et al. (2023); Guo et al. (2022)—to drive full private-key reconstruction.

Finally, we delimit the scope of the present evaluation. The core contribution of this paper is the side-channel sequence-recovery step described above: demonstrating that the CRNN-CTC framework can reliably extract the 16-bit `ctr` branch-outcome sequence from long, temporally unstable BGF decoding traces in a profiled, unprotected, same-platform setting. The paper does not implement the full private-key reconstruction pipeline that would follow from accumulated observations, and accordingly does not claim full key recovery. The reported 99.69% bit accuracy and 95.68% perfect sequence recovery characterize the feasibility and effectiveness of this intermediate leakage-extraction step, which constitutes the technically novel and experimentally validated contribution of the work.

Conclusion

This paper examined the side-channel security of the latest BIKE reference implementation, in which the BGF decoder serves as the recommended decoding procedure. Earlier attacks on BIKE had focused on specific software or hardware versions under controlled acquisition conditions. No previous research has been conducted on the current reference code, nor has the leakage characteristics of the BGF decoding process been investigated under long-tracing settings. The data-dependent nature of the counter computation and conditional bit-flipping operations means that informative leakage is spread unevenly across the acquisition window, which undermines the trace-alignment and fixed-window assumptions that most existing profiling pipelines rely on.

To address this challenge, we reformulated recovery of the private-key-dependent 16-bit sequence produced by counter-threshold comparisons as a sequence-to-sequence problem and validated this alignment-free formulation with a CRNN-CTC instantiation. The model avoids fixed trace partitioning and independent local classification, mapping a complete power trace to the 16-bit target sequence in a single pass. In this framework, the 1D-CNN, Deep BiLSTM, and CTC modules serve as established components for implementing the proposed formulation and assessing its effectiveness on long, temporally unstable BIKE decoding traces.

In the profiled, unprotected and same-platform ARM Cortex-M4 experiment reported here, the proposed method achieved 99.69% bit accuracy and 95.68% perfect sequence recovery, against 0.32% and 5.60% for the rigid-segmentation baselines. The additional No-CTC ablation (Exp. 8), which retains the same CNN-BiLSTM encoder but replaces CTC with frame-wise cross-entropy, achieves only 15.27% perfect recovery, confirming that the 80-percentage-point gain of the full model over Exp. 8 is attributable to the CTC alignment mechanism rather than encoder capacity. The ablation study further showed that removing the Deep BiLSTM module individually causes perfect recovery to collapse to 0, confirming that temporal modeling and CTC-based alignment each address a distinct aspect of the problem rather than providing redundant capacity. These results demonstrate that the studied BGF-based implementation is practically exploitable in this evaluated setting and, more importantly, that alignment-free sequence recovery is an effective formulation for the measured BIKE BGF decoding traces with long, temporally unstable leakage.

Several issues remain unresolved. The present evaluation demonstrates recovery of the private-key-dependent 16-bit

sequence produced by counter-threshold comparisons, but it does not implement the subsequent full-private-key reconstruction stage based on accumulated chosen-ciphertext observations and information-set decoding or combinatorial post-processing. It also does not establish robustness across different devices, acquisition sessions, implementations, keys, or countermeasures. The attack targets an unprotected implementation, and it is unclear whether the same recovery rates would hold in the presence of countermeasures such as masking or instruction shuffling. The profiling requirement also limits deployment flexibility: adapting the framework to non-profiled, cross-session, cross-device, or transfer-learning scenarios would be a natural next step. Beyond BIKE, it would be meaningful to assess whether the alignment-free formulation transfers to other code-based schemes or to FPGA and ASIC implementations of BIKE, where the leakage structure may differ from the software setting studied here.

Acknowledgements
Not applicable.

Author contributions
Geng Chen: Conceptualization, Methodology, Software, Investigation, Formal analysis, Writing – original draft. Yanbin Li: Funding acquisition, Methodology, Supervision, Writing – review & editing. Shilin Sun: Validation, Formal analysis, Data curation. Zongyue Wang: Resources, Investigation, Validation. Chunpeng Ge: Funding acquisition, Supervision, Visualization. Fusheng Wu: Formal analysis, Visualization. Qiuliang Xu: Supervision, Project administration.

Funding
This work was supported by the National Key Research and Development Program of China under Grant 2022YFB4400700.
This work was supported in part by the Program of Taishan Young Scholars of Shandong Province (NO.tsqn202408026), National Natural Science Foundation of China under Grant (NO.62472259, NO.62572285, NO.62172258), Shandong Provincial Natural Science Foundation (ZR2024QF098).

Data availability
The side-channel dataset used in this study will be made available upon reasonable request.

Declarations

Competing interests
The authors declare that they have no conflict of interest.

Received: 24 April 2026 Accepted: 6 September 2026
Published online: 16 September 2026

References

Aragon N, Barreto PSLM, Bettaieb S, Bidoux L, Blazy O, Deneuville J-C, Gaborit P, Ghosh S, Gueron S, Güneysu T, Aguilar Melchor C, Misoczki R, Persichetti E, Richter-Brockmann J, Sendrier N, Tillich J-P, Vasseur V, Zémor G (2024) BIKE: bit flipping key encapsulation, Round 4 Submission. NIST Post-Quantum Cryptography Standardization Project submission (version 5.2)
Beckwith L, Hettwer B, Güneysu T (2024) Power side-channel key recovery attack on a hardware implementation of bike. In: IEEE International Symposium on Hardware Oriented Security and Trust (HOST), pp 267–277. IEEE, Washington, DC, USA.https://doi.org/10.1109/HOST55342.2024.10545264
Benadjila R, Prouff E, Strullu R, Cagli E, Dumas C (2020) Deep learning for side-channel analysis and introduction to the ASCAD database. J Cryptogr Eng 10(2):163–188. https://doi.org/10.1007/s13389-019-00220-8
Cagli C, Eleonora I, Prouff E (2017) Convolutional neural networks with data augmentation against jitter-based countermeasures. Cryptographic Hardware and Embedded Systems—CHES 2017, vol vol 10529. Springer, Cham, pp 45–68. https://doi.org/10.1007/978-3-319-66787-4_3
Cheriere C, Rivain M, Rossi M, Vaslin R (2023) Bike key-recovery: combining power consumption analysis and information-set decoding. In: Tibouchi M, Wang X (eds) Applied cryptography and network security. Springer, Cham, pp 64–84. https://doi.org/10.1007/978-3-031-33491-7_4
Delgado-Vargas KA, Mancillas-López C, Gallegos-García G (2024) A look at side channel attacks on post-quantum cryptography. Comput Sist 28(4):1879–1896
Guo Q, Johansson T, Nilsson A (2022) A key-recovery timing attack on post-quantum primitive BIKE. IEEE Trans Comput 71(5):1144–1153. https://doi.org/10.1109/TC.2021.3075217
Kim J, Picek S, Heuser A, Bhasin S, Hanjalic A (2019) Unleashing the power of convolutional neural networks for profiled side-channel analysis. IACR Trans Cryptogr Hardw Embed Syst 3:148–179. https://doi.org/10.13154/tches.v2019.i3.148-179
Kocher P, Jaffe J, Jun B (1999) Differential power analysis. Advances in Cryptology—CRYPTO '99. Springer, Berlin, Heidelberg, pp 388–397. https://doi.org/10.1007/3-540-48405-1_25
Lu X, Zhang C, Cao P, Gu D, Lu H (2021) Pay attention to raw traces: a deep learning architecture for end-to-end profiling attacks. IACR Trans Cryptogr Hardw Embed Syst 3:235–274. https://doi.org/10.13154/tches.v2021.i3.235-274
Maghrebi H (2019) Deep learning based side channel attacks in practice. Cryptology ePrint Archive, Report 2019/578. https://eprint.iacr.org/2019/578
Maghrebi H, Portigliatti T, Prouff E (2016) Breaking cryptographic implementations using deep learning techniques. In: Security, Privacy, and Applied Cryptography Engineering (SPACE 2016). Lecture Notes in Computer Science, vol 10076, pp 3–26. Springer, Cham.https://doi.org/10.1007/978-3-319-49445-6_1
Picek S, Perin G, Mariot L, Wu L, Batina L (2023) SoK: deep learning-based physical side-channel analysis. ACM Comput Surv 55(11):227–122735. https://doi.org/10.1145/3569577
Richter-Brockmann J, Chen M-S, Ghosh S, Güneysu T (2020) High-performance and scalable hardware architectures for the post-quantum cryptography scheme BIKE. IEEE Trans Comput Aided Des Integr Circuits Syst 40(6):1145–1158
Sendrier N, Vasseur V (2019) On the decoding failure rate of QC-MDPC bit-flipping decoders. In: Post-Quantum Cryptography–10th International Conference, PQCrypto 2019. Lecture Notes in Computer Science, vol 11505, pp 404–416. Springer, Cham. https://doi.org/10.1007/978-3-030-25510-7_22
Timon B (2019) Non-profiled deep learning-based side-channel attacks with sensitivity analysis. IACR Trans Cryptogr Hardw Embed Syst 2019(2):107–131. https://doi.org/10.13154/tches.v2019.i2.107-131
Zaid G, Bossuet L, Habrard A, Venelli A (2020) Methodology for efficient CNN architectures in profiling attacks. IACR Trans Cryptogr Hardw Embed Syst 1:1–36. https://doi.org/10.13154/tches.v2020.i1.1-36

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Yanbin Li Yanbin Li received the Ph.D. degree from the School of Cyber Science and Engineering, Wuhan University. He is currently an Associate Professor with the School of Software, Shandong University, Jinan, China. He was awarded the Taishan Young Scholars of Shandong Province. His research interests include side-channel analysis, cryptographic implementation, hardware security, and applied cryptography.